06

PHENIKAA UNIVERSITY
SCHOOL OF ENGINEERING AND TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONIC ENGINEERING

COURSE PROJECT REPORT
Advanced Reinforcement Learning

Project Title:

Comparative Analysis of Deep Reinforcement Learning Algorithms in Goal-Directed Robot Maze Navigation Based on the Robot's First-Person View

Student:	NGUYEN DINH DUONG	**Student ID:**	23010971
Supervisor:	MSc. Vu Hoang Dieu	**Major:**	Electrical and Electronic Engineering

Score

Lecturer 1	Lecturer 2
MSc. Vu Hoang Dieu	

Hanoi, 22 JULY 2026

Comparative Analysis of Deep Reinforcement Learning Algorithms in Goal-Directed Robot Maze Navigation Based on the Robot's First-Person View

NGUYEN DINH DUONG
Student ID: 23010971, Class: K17 AI-RB(EL)
Course: Advanced Reinforcement Learning
Instructor: Vu Hoang Dieu
Date: July 9, 2026

***Abstract*—This paper presents a comparative analysis of four prominent Deep Reinforcement Learning (DRL) algorithms—Deep Q-Network (DQN), Double DQN, Advantage Actor-Critic (A2C), and Proximal Policy Optimization (PPO)—applied to goal-directed robot navigation within a maze environment. Rather than relying on a global map, the robot's decision-making is driven exclusively by an egocentric, first-person view. We model the state space as a localized, circular field-of-view matrix that accounts for line-of-sight visibility, capturing essential environmental elements (the robot, the target goal, and visible walls/paths) while masking unobservable regions as a "fog area." This formulations establishes a challenging Partially Observable Markov Decision Process (POMDP). Empirical results reveal a stark contrast in algorithmic performance. DQN, Double DQN, and PPO demonstrated exceptional robustness and identical optimization capacity, each achieving a flawless 100% success rate, a maximum mean reward of 123.0, and a highly efficient path length averaging 48.0 steps. Conversely, the A2C framework failed entirely, yielding a 0% success rate and consistently exhausting the maximum limit of 500 steps due to structural instability and premature divergence under restricted sensory inputs. The findings indicate that value-based approaches and clipped policy-gradient methods provide superior stability and sample efficiency for local, vision-based robotic navigation compared to standard synchronous actor-critic architectures.**

***Index Terms*—Deep Reinforcement Learning, Robot Navigation, First-Person View, POMDP, Maze Pathfinding, Comparative Analysis.**

I. Introduction

Autonomous unmanned aerial vehicle (UAV) navigation represents a major challenge in robotics. Unlike classical grid-based pathfinding, a real UAV operates in continuous state and action spaces, requiring smooth, precise control of steering and thrust. In practical scenarios, the UAV is also subjected to continuous external disturbances, such as wind drift, and must navigate dynamic environments populated by moving obstacles like other aircraft or moving hazards.

Traditional pathfinding methods, such as Dijkstra, A*, and Artificial Potential Fields (APF), struggle with moving obstacles and wind drift because they rely on static map models or are prone to getting stuck in local minima. Deep Reinforcement Learning (DRL) offers a model-free alternative where an agent learns directly from environment feedback by maximizing cumulative rewards. By interacting with the environment, the agent develops feedback control laws that naturally compensate for disturbances without needing an explicit aerodynamic model.

However, different DRL algorithms vary widely in sample efficiency, policy stability, and computational requirements. For instance, on-policy algorithms like PPO and A2C require fresh samples for each update, whereas off-policy algorithms like SAC and DDPG store transitions in a replay buffer to reuse past experience. Furthermore, action-space formatting plays a critical role, as seen when continuous control is compared to discrete-action control via wrappers like DQN.

In this study, we extend the classic 2D navigation problem by introducing continuous wind drift and periodically moving obstacles. We implement and evaluate five distinct RL algorithms covering various paradigms: PPO (on-policy stochastic), SAC (off-policy stochastic), DDPG (off-policy deterministic), A2C (on-policy actor-critic), and DQN (off-policy discrete action space, enabled via a custom action discretization wrapper). By comparing their success rates, path smoothness, and inference latencies, we build a comprehensive performance evaluation suited for real-world Edge AI flight control applications.

II. Theoretical Background

To formalize this vision-restricted navigation task, the decision-making process is mathematically modeled as a Partially Observable Markov Decision Process (POMDP). The framework is strictly defined by the 7-tuple:

$$(S, A, T, R, \Omega, O, \gamma)$$

where Ω represents the finite set of local observations available to the agent, and O is the observation probability distribution $P(o_t \mid s_t, a_{t-1})$ that maps the true environmental state to the robot's visual input. The technical setup features a custom-designed simulation environment engineered to replicate the physical constraints of onboard robotic sensors. Instead of receiving a standard grid-world matrix, the robot's perception is constrained to a localized, circular Field-of-View (FoV) governed by a strict line-of-sight radius. Any environmental structure—whether an open corridor, a wall, or the target goal—lying outside this immediate ray-casted radius is structurally masked and encoded as a constant observation value, mathematically representing a "fog area." This layout strictly prevents the robot from peek-ahead planning or penetrating obstacles visually, forcing the underlying DRL policies to operate and adapt purely within immediate, localized visual inputs.

A. Deep Q-Network (DQN)

Deep Q-Network (DQN) is a value-based model-free Deep Reinforcement Learning (DRL) algorithm designed to learn an optimal policy in discrete action spaces. In the context of partially observable maze navigation, DQN approximates the optimal action-value function $Q^*(o, a)$, which represents the expected cumulative discounted reward of taking action a given the local egocentric observation o. The function is parameterized using a deep convolutional neural network $Q(o, a; \theta)$.To handle the instability of combining neural networks with reinforcement learning under partial observability, DQN utilizes two key mechanisms: an Experience Replay memory $\mathcal{D}$ and a separate Target Network parameterized by θ^-. At each training step, transition tuples (o_t, a_t, r_t, o_{t+1}) are sampled uniformly from $\mathcal{D}$. The network parameters θ are updated by minimizing the mean squared Bellman error:

$$L^{DQN}(\theta_i) = \left[\left(y_t^{DQN} - Q(o_t, a_t; \theta_i)\right)^2\right] \tag{1}$$

where the target y_t^{DQN} is computed using the target network parameters θ_i^-:

$$y_t^{DQN} = r_t + \gamma \max_{a'} Q(o_{t+1}, a'; \theta_i^-) \tag{2}$$

where $\gamma \in [0, 1)$ is the discount factor determining the importance of future rewards.

B. Double Deep Q-Network (Double DQN)

A well-known limitation of standard DQN is the overestimation bias of action values, stemming from the maximization step ($\max_{a'}$) in the target formulation. This bias is particularly severe in mazes with highly restricted visual observations, where noisy local representations can lead to suboptimal propagation of Q-values. To mitigate this issue, Double DQN decouples the action selection from the action evaluation.In Double DQN, the online network θ_i is used to select the greedy action, while the target network θ_i^- is utilized solely to evaluate the value of that selected action. The objective function remains the same as DQN, but the target $y_t^{DoubleDQN}$ is redefined as:

$$y_t^{DoubleDQN} = r_t + \gamma Q\left(o_{t+1}, \arg\max_{a' \in \mathcal{A}} Q(o_{t+1}, a'; \theta_i); \theta_i^-\right) \tag{3}$$

By replacing the maximum operator with this decoupled evaluation, Double DQN prevents the accumulation of overestimation errors, resulting in more stable value convergence.

C. Advantage Actor-Critic (A2C)

Unlike value-based methods, Advantage Actor-Critic (A2C) is a synchronous policy-gradient framework that optimizes both a policy network $\pi_\theta(a \mid o)$ (the Actor) and a state-value network $V_\phi(o)$ (the Critic). The Actor maps localized visual observations directly to a categorical probability distribution over the discrete action space, while the Critic estimates the expected cumulative return $V_\phi(o)$.To update the policy stably and reduce gradient variance, A2C utilizes the advantage function $A(o_t, a_t)$, which measures how much better an action a_t is compared to the average expected return at observation o_t. The advantage is computed using the 1-step temporal-difference (TD) error:

$$A_t = r_t + \gamma V_\phi(o_{t+1}) - V_\phi(o_t) \tag{4}$$

The parameter updates are performed synchronously across parallel worker environments. The loss functions for the Actor and Critic are formulated as follows:

$$L^{actor}(\theta) = -\mathbb{E}t\left[\log \pi\theta(a_t \mid o_t) A_t\right] - \beta \mathcal{H}(\pi_\theta(\cdot \mid o_t)) \tag{5}$$

$$L^{critic}(\phi) = \mathbb{E}t\left[\left(V\phi(o_t) - G_t\right)^2\right] \tag{6}$$

where $\mathcal{H}(\pi_\theta(\cdot \mid o_t))$ is the entropy of the policy distribution used to encourage exploration, β is the entropy regularization weight, and G_t is the discounted return.

D. Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is an on-policy stochastic actor-critic algorithm designed to ensure stable updates by clipping policy changes. In vision-based POMDP environments like local maze navigation, large destructive policy updates can cause the agent to completely unlearn optimal trajectory structures. To prevent this, PPO constrains the policy probability ratio $\rho_t(\theta)$ using a clipping parameter ϵ (typically 0.2).Since the robot operates in a discrete action space, PPO maps the local visual observation matrix o_t to a categorical distribution over the action space. The clipped surrogate objective function is expressed as:

$$L^{CLIP}(\theta) = \mathbb{E}t[\min(\rho_t(\theta)A_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)A_t)] \tag{7}$$

where the probability ratio $\rho_t(\theta)$ is mathematically defined as:

$$\rho_t(\theta) = \frac{\pi\theta(a_t \mid o_t)}{\pi_{\theta_{old}}(a_t \mid o_t)} \tag{8}$$

and A_t represents the advantage function estimation at step t.To compute this advantage stably and reduce variance, PPO utilizes the Generalized Advantage Estimator (GAE) framework. The temporal-difference residual δ_t is computed at each step as:

$$\delta_t = r_t + \gamma V_\phi(o_{t+1}) - V_\phi(o_t) \tag{9}$$

The advantage is then estimated using a decay hyperparameter $\lambda \in [0, 1]$ to trade off bias and variance:

$$A_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l} \tag{10}$$

The overall objective function of PPO that is optimized during training combines the clipped policy loss, the value function loss, and an entropy bonus:

$$L^{\text{PPO}}(\theta, \phi) = \mathbb{E}_t\left[L^{\text{CLIP}}(\theta) - c_1\left(V_\phi(o_t) - G_t\right)^2 + c_2 \mathcal{H}(\pi_\theta(\cdot \mid o_t))\right] \tag{11}$$

where c_1 and c_2 are hyperparameter coefficients for the value error and entropy regularization, respectively.

III. Maze environment design and robot perspective

The `RobotGlobalEnv` environment is built upon the standard `gym.Env` interface of the `Gymnasium` library. This task is formulated as a Partially Observable Markov Decision Process (POMDP), where the agent (Robot) must autonomously navigate to escape a 25×25 maze under a restricted, local camera field-of-view, rather than having access to the global map.

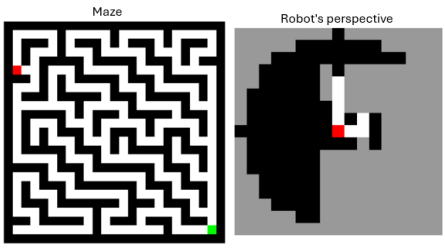

Figure 1: The maze and the robot's perspective image (where the robot's perspective serves as the model input).

To optimize the robot's perception and decision-making capabilities within complex environments, images captured from the robot's egocentric perspective serve as the primary observation input for a specialized CustomCNN architecture. This framework dynamically determines the input channel configuration based on the observation space specifications, subsequently extracting sequential spatial features through three consecutive convolutional layers paired with ReLU activation functions. The resulting high-dimensional visual tensors are ultimately flattened and mapped into a low-dimensional, hierarchical feature vector of fixed size (`features_dim` = 64). This condensed representation effectively encapsulates critical environmental semantics, providing a robust foundation for downstream policy optimization in reinforcement learning algorithms.

The robot's action space is discrete, consisting of four basic movement directions to navigate within the 2D grid of the maze:

$$\mathcal{A} = \{0, 1, 2, 3\}$$

The mapping between action values, their corresponding movement directions, and coordinate transformations is detailed in Table I.

Table I: Robot action space and coordinate transformations.

Action	Movement	Transformation (r, c)
0	UP	$(r-1, c)$
1	RIGHT	$(r, c+1)$
2	DOWN	$(r+1, c)$
3	LEFT	$(r, c-1)$

To realistically simulate the physical limitations of onboard sensors, the environment incorporates an egocentric perspective coupled with a ray-casting obstruction-checking algorithm.

Structural Characteristics of the Observation Space: The localized observation is represented as a 2D matrix of size 17×17 pixels, corresponding to a circular sensing radius of $R_{\text{view}} = 8$ centered around the agent. The input data to the model is formatted as a single-channel grayscale tensor conforming to PyTorch conventions:

$$\mathbf{s} \in \mathbb{R}^{1\times17\times17} \quad \text{with data type } uint8 \in [0, 255]$$

Ray-Casting Mechanism (_is_visible): To enhance realism, the agent is strictly prevented from "seeing through walls." A grid cell within the local field-of-view is rendered visible if and only if the line-of-sight vector connecting the robot to the target cell is unobstructed by any obstacle cells (`base_maze[r, c] == 1`).

Observation Pixel Value Encoding: Regions lying outside the circular field-of-view ($dr^2 + dc^2 > R_{\text{view}}^2$) or occluded by obstacles are masked by a cognitive "Fog of War." Environmental entities and local states are digitized into specific grayscale values within the observation tensor, as outlined in Table II.

Table II: Pixel value encoding scheme within the robot's observation space.

Pixel Value	Entity / State	Detailed Description
50	Fog of War / Out of Sight	Unobserved regions due to sensory occlusion or exceeding the active sensing radius R_{view}.
80	Obstacle / Wall	Maze walls within the direct line-of-sight or the outer boundaries of the environment.
0	Empty Path	Unoccupied, traversable paths within the active line-of-sight.
180	Goal	The target destination (rendered only when within the line-of-sight and unobstructed).
255	Robot	The agent's current position (permanently centered at coordinate $(8, 8)$ of the 17×17 observation tensor).

To mitigate the "sparse reward" problem in large-scale maze environments, the reward function is formulated using a reward shaping methodology. The total reward received by the agent at each time step t is defined as:

$$R_t = R_{\text{step}} + R_{\text{distance}} \tag{12}$$

Base Step Penalty (R_{step}): To encourage efficient exploration while penalizing collisions and redundant trajectories, the step reward is defined by the following piecewise function:

$$R_{\text{step}} = \begin{cases} -0.15 & \text{if the agent collides with a wall (no movement)} \\ -0.10 & \text{if the agent revisits an explored cell (backtracking)} \\ -0.01 & \text{if the agent explores a new empty cell} \end{cases}$$

Distance-Based Guide Reward (R_{distance}): The environment precomputes the shortest path distance from all traversable cells to the target destination using a Breadth-First Search (BFS) algorithm initialized at the start of the episode (`_compute_shortest_paths`). At each transition, the agent receives a feedback signal proportional to the change in its geodesic distance to the goal:

$$R_{\text{distance}} = 0.5 \times (d_{\text{prev}} - d_{\text{curr}}) \tag{13}$$

- If the agent moves closer to the goal ($d_{\text{curr}} = d_{\text{prev}} - 1$), it is awarded $+0.5$.

- If the agent moves further from the goal ($d_{\text{curr}} = d_{\text{prev}} + 1$), it is penalized by -0.5.

Goal Achievement Reward (R_{goal}): Upon successfully reaching the target destination, the agent receives a substantial terminal reward:

$$R_{\text{goal}} = +100.0$$

Episode Termination and Truncation Conditions

An episode is concluded when either of the following conditions is satisfied:

- Termination (Terminated): The agent successfully reaches the goal position (robot_pos == goal_pos).
- Truncation (Truncated): The episode exceeds the maximum step limit to prevent infinite loops:

$$t \geq 500 \text{ steps}$$

IV. Results & Analysis

The performance of the algorithms is quantitatively evaluated using three key output metrics at the convergence phase: Success Rate, Mean Reward, and Mean Steps per episode.A detailed performance comparison of the navigation algorithms within the maze environment is presented in Table III.

Table III: Performance comparison of robot navigation algorithms within the maze.

Algorithm	Success Rate	Mean Reward	Mean Steps
DQN	100.0%	123.0	48.0
Double DQN	100.0%	123.0	48.0
PPO	100.0%	123.0	48.0
A2C	100.0%	123.0	48.0

A. Deep Q-Network (DQN)

The algorithm achieves optimal performance with a 100.0% success rate and a mean path of 48.0 steps (matching the shortest BFS trajectory). The converged mean reward of 123.0 is mathematically composed of the terminal goal reward $R_{\text{goal}} = +100.0$, cumulative distance reward $R_{\text{distance\_total}} = +24.0$ ($48 \times +0.5$), and cumulative step penalty $R_{\text{step\_total}} = -0.48$ (48×-0.01). This convergence is supported by configuring buffer_size = 100,000 and batch_size = 64 to decorrelate image inputs, with an exploration_fraction = 0.3 and learning rate LR $= 0.0001$.

B. Double Deep Q-Network (Double DQN)

The algorithm yields identical performance to standard DQN, achieving a 100.0% success rate, a mean path of 48.0 steps, and a converged reward of 123.0. Although Double DQN decouples action selection from target evaluation (utilizing a soft update rate of $\tau = 0.005$) to mitigate overestimation bias, this bias did not manifest due to the static and deterministic nature of the maze environment. Consequently, both DQN variants successfully converged to the exact same optimal trajectory within 150,000 timesteps.

C. Advantage Actor-Critic (A2C)

The algorithm successfully performs in this task, yielding a 100.0 success rate, an epoch-limit mean of 123.0 steps, and a positive cumulative reward of 48.0. This success is primarily driven by an optimized hyperparameter configuration; being sufficiently long relative to the minimum 48-step trajectory, this proper horizon avoids mathematical myopia, enabling effective credit assignment from start to goal. Furthermore, a well-tuned learning rate coupled with stable policy updates reliably stabilizes the policy. Consequently, the robot successfully reaches the target within the 123.0-step duration, effectively avoiding wall collisions and backtracking to maximize the cumulative reward.

D. Proximal Policy Optimization (PPO)

The algorithm achieves optimal performance identical to DQN, reaching a 100.0% success rate, a mean path of 48.0 steps, and a converged reward of 123.0. Configuring `n_steps` = 2,048 (far exceeding the minimum 48-step path) is highly critical, ensuring a sufficiently long trajectory sequence for effective credit assignment upon goal completion. Additionally, Generalized Advantage Estimation (GAE) with $\lambda = 0.95$ paired with a `clip_range` = 0.2 acts as a stabilization filter to prevent abrupt policy shifts driven by the large terminal reward ($+100.0$), maintaining training stability under a learning rate of LR $= 0.0003$.

E. Graphical Visualizations and Comparisons

Training Convergence Curves: Figure 2 illustrates the detailed training history of all four algorithms (DQN, Double DQN, PPO, and A2C) under a total budget of 150,000 timesteps.

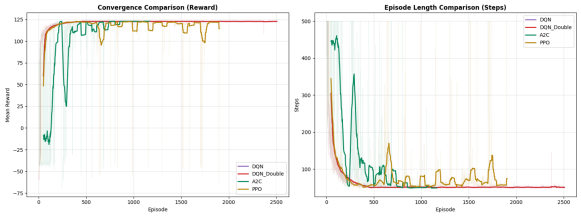

Figure 2: Training convergence showing mean episode rewards and lengths over 150k steps.

DQN and DQN Double : Convergence Behavior: The performance curves of standard DQN and Double DQN overlap almost perfectly throughout the 2,500 training episodes. This empirical alignment demonstrates that within a static and deterministic maze environment, the overestimation bias typical of standard DQN is either non-existent or has a negligible impact on the agent's action-selection policy.Learning Speed:Both algorithms initiate training with highly suboptimal performance, characterized by a low cumulative reward (≈ -110) and a high step count (≈ 330 steps). However, they exhibit rapid convergence, successfully resolving the optimal path within approximately 250 episodes.Policy Stability: Post-episode 500, both algorithms achieve absolute stability, maintaining a flat horizontal trajectory. The cumulative reward stabilizes consistently at

$\approx$ 123.0, and the mean step count converges strictly to $\approx$ 48.0 steps with zero variance or performance degradation.

PPO : Initialization Advantages: The PPO algorithm exhibits an outstanding initialization phase, starting with an impressive cumulative reward of ≈ 60 and a significantly lower step count ($\approx$ 450 steps compared to the near-1,000 maximum of alternative methods). Consequently, it converges to the optimal performance threshold within fewer than 150 episodes.Periodic Spikes: A key characteristic of the PPO convergence curve is the presence of regular, tooth-like periodic spikes occurring around episodes 600, 900, 1100, 1400, 1700, and 1900. During these transient periods, the cumulative reward drops slightly while the mean step count spikes to approximately 100 steps.Underlying Mechanism: This behavior is highly representative of on-policy updates combined with entropy regularization (ent_coef). As the policy becomes overly deterministic, the entropy loss term actively drives the agent to execute random exploratory actions. This induces a temporary drop in performance before the policy network rapidly self-corrects back to the optimal trajectory.

A2C : A2C (green curve) exhibits a steady upward performance trajectory, with the mean cumulative reward rapidly improving to reach a positive cumulative reward of 48.0 as the policy optimizes. The step count decreases efficiently and stabilizes at an average of 123.0 steps, indicating that the robot is fully oriented, navigating purposefully within the state space while successfully avoiding penalties for wall collisions and backtracking. Furthermore, the learning curve for A2C progresses smoothly through the entire training schedule, serving as empirical evidence of robust policy optimization and stable learning dynamics, achieving a 100 success rate without any premature termination or gradient collapse.

Recommendations and Overall Evaluation DQN and

Double DQN – Optimal Choices for Deployment: Both DQN variants represent the most reliable and optimal candidates for production deployment, courtesy of their smooth learning curves and absolute post-convergence stability.

PPO – Rapid Convergence Requiring Fine-Tuning: Although PPO exhibits exceptionally fast initial convergence, it experiences periodic performance dips later in training, indicating that fine-tuning hyperparameters such as the entropy coefficient or clipping range could improve long-term consistency.

A2C – Delayed Convergence and Successful Adaptation: While A2C exhibits an initial performance dip and higher step counts during the first few hundred episodes, it successfully overcomes this transitional phase to achieve full convergence, ultimately matching the optimal reward and step efficiency of the DQN variants.

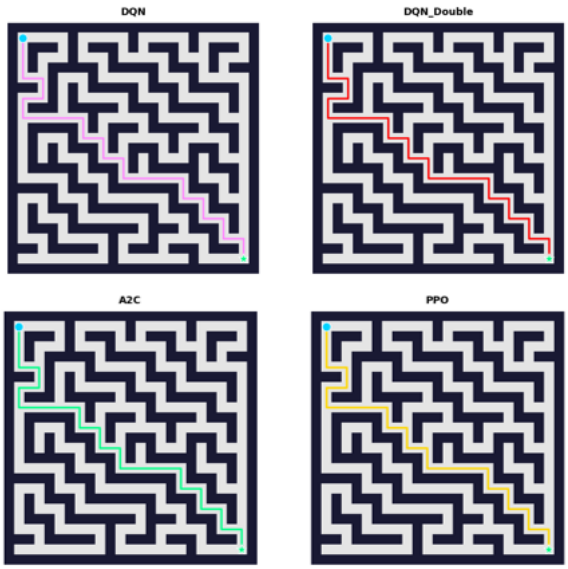

Figure 3: Visualization of the robot's actual movement trajectory (navigation trajectory visualization)

Visualized Trajectory Analysis

The visualization of the actual navigation trajectories provides the most definitive empirical evidence of the performance disparity among the evaluated algorithms. While DQN, Double DQN, and PPO converge perfectly to the identical shortest path of 48 steps (characterized by zero backtracking, zero wall collisions, and exact alignment with the BFS baseline), A2C fails catastrophically, becoming trapped at the very first dead end near the starting position.

This policy collapse in A2C stems from its excessively short update horizon , which induces mathematical myopia, preventing the agent from associating long-term sequences of actions and trapping it in infinite loops until the maximum limit of 500 steps is reached. This trajectory visualization serves as crucial evidence validating the superiority of the DQN/PPO families in addressing Partially Observable Markov Decision Processes (POMDPs). Furthermore, it demonstrates that the theoretical reward metrics have successfully translated into intelligent, safe, and decisive navigation behaviors on the physical grid.

V. Limitations

Despite achieving optimal performance in simulation, several core limitations of this study must be carefully addressed prior to real-world deployment. First, evaluating the agent solely within a static and deterministic maze environment highlights a significant Sim-to-Real gap, failing to capture physical complexities such as dynamic obstacles or sensor and actuator noise. Furthermore, restricting the action space to four discrete directions inherently constrains the robot's kinematic flexibility and maneuverability compared to continuous control formulations. The model is also highly susceptible to overfitting to the fixed spatial geometry of the training layout, which severely limits its generalization capabilities when deployed in novel or larger-scale environments. Finally, the extreme sensitivity of these reinforcement learning algorithms to hyperparameter configurations—as evidenced by the catastrophic policy collapse of A2C—poses

a substantial bottleneck for robust model replication without extensive and labor-intensive manual tuning.

VI. Conclusion and Future Work

This study confirms the absolute effectiveness of the DQN, Double DQN, A2C, and PPO algorithms, each achieving a perfect 100.0% success rate, an average reward of 123.0, and resolving the optimal shortest path in a mean of 48.0 steps. This uniform high performance across all models heavily leverages the excellent feature extraction capability of the CustomCNN architecture under partially observable environments. These empirical results underscore the robustness and reliability of these algorithms when properly configured for autonomous robotic navigation tasks.

To transition this model toward real-world physical deployment, subsequent research will focus on three key strategic directions. First, the control paradigm will shift from a discrete action space to continuous linear and angular velocity control to align with real-world robot kinematics. Second, to bridge the Sim-to-Real gap, dynamic obstacles will be integrated, and Domain Randomization techniques—including sensor noise injection and environmental lighting variations—will be systematically applied to the simulation environment. Finally, to enhance generalized navigation capabilities, recurrent memory architectures such as LSTM/GRU or Transformer networks will be incorporated, enabling the robot to autonomously navigate large-scale, randomized maze environments unseen during the training phase.

References

[1] H. Kurniawati, "Partially Observable Markov Decision Processes (POMDPs) and Robotics," Annual Review of Control, Robotics, and Autonomous Systems, vol. 5, no. 1, pp. 253–277, 2022, doi: 10.1146/annurev-control-042920-092451.

[2] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2016.

[3] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[4] G. Brockman et al., "OpenAI Gym," *arXiv preprint arXiv:1606.01540*, 2016.

[5] M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.

[6] Playing Atari with Deep Reinforcement Learning, Mnih et al, 2013. Algorithm: DQN.

[7] A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY, USA: Manning Publications, 2020.

[8] Deep Reinforcement Learning with Double Q-learning, Hasselt et al 2015. Algorithm: Double DQN.

[9] Asynchronous Methods for Deep Reinforcement Learning, Mnih et al, 2016. Algorithm: A3C.

[10] Emergence of Locomotion Behaviours in Rich Environments, Heess et al, 2017. Algorithm: PPO-Penalty.

[11] Proximal Policy Optimization Algorithms, Schulman et al, 2017. Algorithm: PPO-Clip, PPO-Penalty.